

WILDTRACK CONSERVATION ALLIANCE

Wildlife Conservation Database & SQL Analysis

A Relational Database Project Applying SQL to Real-World Wildlife Conservation Operations

SQL / DATABASE PROJECT

Project Report

Author

Prativa Baliarsingh

2. Executive Summary

The WildTrack Conservation Alliance database exists to give a multi-country wildlife conservation programme a single, reliable source of truth for everything it protects and monitors: forest reserves, the species living in them, the individual animals fitted with GPS collars, and the continuous stream of location pings those collars send back. Field rangers, biologists, the data team, and donor relations all depend on this database daily — to plan patrols, catch failing equipment before an animal goes dark, spot signs of poaching or distress, and report results back to the funders keeping the programme running.

GPS tracking data matters to wildlife conservation because it converts an animal's movement, health, and location into something that can be measured, queried, and acted on. A single missed ping can mean nothing, but a pattern of missed pings across a species or a reserve can mean a communication blackout, a hardware failure, or an emerging threat on the ground. Turning that raw stream of pings into timely, structured insight is the core value this project demonstrates.

This SQL project demonstrates the ability to design a normalized relational schema, populate and query it at meaningful scale, and translate everyday operational questions — asked in plain business language — into precise SQL that a live conservation programme could actually run. It covers the full spectrum of practical SQL skill: schema design and constraints, joins and aggregation, subqueries and CTEs, window functions, views, indexing, data modification, and role-based security.

The database is built around four connected tables:

- Reserves — every forest reserve or national park in the network
- Species — every species currently being monitored, along with its conservation status
- Animals — every individual animal fitted with a collar, linked to its species and its home reserve
- GPSLogs — every single location ping sent back by a collar, linked to the animal wearing it

As stated in the source material, the database holds 10,000 records across these four tables, the vast majority of them GPS pings, since a handful of collars quietly generate an enormous stream of location data every day. The uploaded SQL file confirms this scale precisely: 15 reserves, 25 species, 200 animals, and 9,760 GPS log entries — a total of 10,000 rows, matching the scenario description exactly.

The 75 business cases solved in this project fall into major categories: data retrieval and aggregation, data quality and validation, animal and species analysis, GPS and movement analysis, battery and collar monitoring, reserve-level analysis, date/time analysis, advanced SQL and window functions, views, indexing and performance, constraints, data modification, and database security. Together they show SQL used not as an academic exercise, but as the operational backbone of a real conservation programme.

3. Project Background

WildTrack Conservation Alliance is described in the source scenario as a multi-country wildlife conservation programme protecting endangered species across a network of forest reserves and national parks. Field rangers fit GPS tracking collars onto individual animals — tigers, elephants, snow leopards, river dolphins, and dozens of other threatened species — and each collar quietly beams back its location, speed, temperature, and battery health every few hours, day and night.

Over the past few years this has grown into a live operational database that several distinct groups depend on:

- Field rangers — use the data to plan patrols and respond quickly when an animal's collar goes silent or shows signs of unusual movement.
- Biologists — study population structure, age distribution, species concentration, and movement/behaviour patterns.
- The data team — maintains data quality, database performance, and the underlying schema as the programme scales.
- Donor relations — reports programme results (species coverage, at-risk populations, reserve activity) back to the funders sustaining the work.

GPS collars generate operational data automatically and continuously: every few hours each collar transmits a location, a speed reading, a temperature reading, and its current battery level. At the scale described in the source material — 200 animals generating 9,760 individual pings — this produces a genuinely large, fast-growing operational table (GPSLogs) that must be queried efficiently and interpreted correctly.

A reliable database is critical here because the underlying subject matter has real consequences: a malfunctioning collar can mean an animal is effectively lost to monitoring; a coverage gap at a reserve can mean poaching or distress goes undetected; and a batch of dirty or duplicate data can quietly corrupt every downstream report. The database is the single mechanism that turns a flood of raw pings into decisions rangers, biologists, and donors can actually act on.

4. Business Problem

Running a programme like WildTrack surfaces a recurring set of operational problems that this project addresses directly through the database and its SQL layer:

- Wildlife tracking — knowing, at any moment, which animals are being monitored, where, and by which reserve.
- GPS data monitoring — turning a continuous ping stream into usable movement and behaviour signals.
- Collar battery health — identifying failing or ageing hardware before an animal goes dark.
- Missing GPS reports — detecting animals that have stopped reporting, whether briefly or entirely.
- Reserve coverage — confirming every reserve in the network is actually being monitored, not just nominally protected.
- Species monitoring — tracking population counts and concentration per species, especially the most vulnerable ones.
- Animal activity — distinguishing normal movement from unusual speed or behaviour that may signal stress, a chase, or a poaching event.
- Data quality — correcting inconsistent capitalization, stray whitespace, invalid values, and duplicate entries introduced by manual, distributed data entry.
- Database security — ensuring only the right people (researchers, veterinarians, field staff) can see or change the right data.
- Database performance — keeping lookups fast as the GPSLogs table grows into the thousands and beyond.
- Data maintenance — keeping collar records, reserve assignments, and battery calibrations accurate as circumstances change on the ground.

5. Project Objectives

- Manage the core wildlife conservation dataset across reserves, species, animals, and GPS logs in a single relational schema.
- Monitor individual animals and their GPS log history to understand movement and reporting patterns.
- Identify inactive, missing, or under-reporting animals before they become an operational blind spot.
- Monitor collar battery health and hardware age to plan proactive maintenance.
- Analyze species population and reserve-level coverage to support conservation prioritization.
- Improve data quality by cleaning inconsistent, duplicate, or malformed values.
- Improve database performance through targeted indexing as the dataset scales.
- Apply database security through role-based access for researchers, veterinarians, and field staff.
- Generate operational insights that translate directly into ranger, biologist, and donor-relations decisions.

6. Database Architecture

The schema, exactly as defined in the uploaded SQL file, consists of four tables. Each is described below with its business purpose, the information it stores, how it connects to the others, and its keys and constraints as actually written in the source SQL.

6.1 Reserves

Business purpose: represents every forest reserve or national park in the WildTrack network — the physical locations the programme protects.

Column	Type	Constraint
reserve_id	INT	PRIMARY KEY
reserve_name	VARCHAR(80)	NOT NULL, UNIQUE
country	VARCHAR(50)	NOT NULL
area_sq_km	INT	CHECK (area_sq_km > 0)
established_year	INT	CHECK (established_year >= 1900)

The uploaded SQL file populates this table with 15 reserves, all located in India (e.g. Jim Corbett National Park, Kaziranga National Park, Ranthambore National Park), with area and establishment-year values used throughout the 75 cases.

6.2 Species

Business purpose: represents every species currently being monitored by the programme, along with its conservation status — the classification driving most conservation-priority analysis.

Column	Type	Constraint
species_id	INT	PRIMARY KEY
species_name	VARCHAR(60)	NOT NULL, UNIQUE
conservation_status	VARCHAR(30)	NOT NULL
average_lifespan_years	INT	CHECK (average_lifespan_years > 0)

The source SQL populates 25 species with conservation_status values including Endangered, Vulnerable, Critically Endangered, Near Threatened, and Least Concern — statuses referenced repeatedly across the 75 cases (e.g. Cases 10, 22, 25, 56).

6.3 Animals

Business purpose: represents every individual animal fitted with a GPS collar — the operational unit the entire programme is built around.

Column	Type	Constraint
animal_id	INT	PRIMARY KEY
tag_code	VARCHAR(20)	NOT NULL, UNIQUE

Column	Type	Constraint
species_id	INT	FOREIGN KEY → Species(species_id)
reserve_id	INT	FOREIGN KEY → Reserves(reserve_id)
sex	VARCHAR(10)	CHECK (sex IN ('Male','Female','Unknown'))
estimated_age	INT	CHECK (estimated_age >= 0)
collar_fitted_date	DATE	NOT NULL

200 animal records are present in the source data. Two additional columns are introduced later in the project via ALTER TABLE statements while solving specific cases: Collar_Manufacturer (Case 38) and collar_model (Case 50).

6.4 GPSLogs

Business purpose: represents every single location ping sent back by a collar — by far the largest and fastest-growing table, and the primary source of movement, speed, and battery-health signal.

Column	Type	Constraint
log_id	INT	PRIMARY KEY
animal_id	INT	FOREIGN KEY → Animals(animal_id)
log_datetime	DATETIME	NOT NULL
latitude / longitude	DECIMAL(9,6)	—
speed_kmph	DECIMAL(5,2)	CHECK (speed_kmph >= 0)
temperature_c	DECIMAL(4,1)	—
battery_level_pct	INT	CHECK (battery_level_pct BETWEEN 0 AND 100)

9,760 GPS log rows are present in the source data — consistent with the scenario's statement that a handful of collars generate the overwhelming majority of the programme's records.

6.5 Entity-Relationship Overview

```

Reserves (1) ----< Animals (many)
  reserve_id          reserve_id (FK)

Species (1) -----< Animals (many)
  species_id          species_id (FK)

Animals (1) -----< GPSLogs (many)
  animal_id           animal_id (FK)

```

Every animal belongs to exactly one reserve and exactly one species; every GPS log belongs to exactly one animal. This is a classic one-to-many chain (Reserve/Species → Animal → GPSLog), which is why so many of the 75 cases combine two or three JOINS to move information from GPSLogs back up to the reserve or species level.

7. Data Model / Relationships

Reserve → Animals

Each reserve can host any number of animals (or zero, as tested directly in Case 3). This relationship is the basis of every reserve-level rollup in the project — animal counts, average age, average battery health, and monitoring-density classification (Cases 3, 21, 31, 42, 44).

Species → Animals

Each species can have any number of tracked individuals across any number of reserves. This relationship drives population-level conservation analysis — counting individuals per species, flagging species with dangerously small tracked populations, and cross-referencing conservation status against reserve location (Cases 10, 22, 25, 47, 56, 72).

Animal → GPSLogs

Each animal can have any number of GPS logs — from zero (a critical failure case, see Case 36) to thousands. This relationship is the foundation of every movement, speed, battery-trend, and reporting-gap analysis in the project (Cases 2, 7, 8, 19, 26, 32, 36, 48, 60, 62, 65, 68, 70).

These relationships matter for analysis because they let a single query travel from the most granular record in the system (one GPS ping) all the way up to the most strategic one (a reserve-level or alliance-level summary), which is exactly what the 75 business cases require.

8. SQL Skills and Concepts Used

The techniques below were identified directly from the uploaded SQL file. Only techniques actually present in the source are listed.

A. DDL (Data Definition Language)

- CREATE DATABASE — Project_2
- CREATE TABLE — Reserves, Species, Animals, GPSLogs, all with explicit column types and constraints
- ALTER TABLE ... ADD CONSTRAINT — new CHECK and UNIQUE constraints added post-creation (Cases 18, 39, 58)
- ALTER TABLE ... ADD COLUMN — schema extension (Cases 38, 50)
- ALTER TABLE ... MODIFY — tightening an existing column to NOT NULL (Case 51)
- CREATE VIEW / DROP VIEW — Reserve_Animal_Battery_level, animal_latest_snapshot, Critically_endangered_animals (Cases 14, 55, 56)
- CREATE INDEX — single-column and composite indexes (Cases 28, 53)
- CREATE USER / DROP USER — account lifecycle management (Cases 1, 69, 71)

B. DML (Data Manipulation Language)

- INSERT INTO — used to load all reference/sample data, and in later cases to demonstrate constraint behaviour (Cases 58, 64)
- UPDATE ... SET ... WHERE — single-row and bulk corrections (Cases 29, 45 scenario, 54, 61)
- DELETE ... (with JOIN) ... WHERE — targeted deletion of obsolete records (Case 50)
- SELECT — used throughout, from simple lookups to multi-table analytical queries

C. Constraints

Constraint	Where used in source
PRIMARY KEY	reserve_id, species_id, animal_id, log_id
FOREIGN KEY	Animals.species_id → Species; Animals.reserve_id → Reserves; GPSLogs.animal_id → Animals
NOT NULL	reserve_name, country, species_name, conservation_status, tag_code, collar_fitted_date, log_datetime; species_id tightened to NOT NULL in Case 51
UNIQUE	reserve_name, species_name, tag_code; composite UNIQUE(tag_code, collar_fitted_date) added in Case 39
CHECK	area_sq_km > 0; established_year >= 1900; average_lifespan_years > 0; sex IN ('Male', 'Female', 'Unknown'); estimated_age >= 0 (defined at creation, re-added in Case 58); speed_kmph >= 0; battery_level_pct BETWEEN 0 AND 100 (defined at creation, re-added in Case 18)
DEFAULT	Not provided in the source data

D. Data Retrieval & Aggregation

- WHERE — filtering throughout (e.g. Cases 5, 11, 20, 24)
- GROUP BY — Cases 2, 10, 13, 21, 25, 31, 42, 43, 52, 67
- HAVING — Cases 24, 25, 42, 43
- ORDER BY — Cases 27, 35, 40, 41, 46, 49, 52, 60, 63, 66
- COUNT / COUNT(DISTINCT) — Cases 2, 4, 14, 31, 37, 42, 44, 57
- SUM — Case 49 (Total_activity)
- AVG — Cases 7, 22, 24, 33, 43, 44, 52
- MIN / MAX — Cases 8, 12, 19, 27, 40, 41, 60
- DISTINCT — Cases 4, 65

E. Joins

- INNER JOIN — the most common join in the project (e.g. Cases 2, 10, 17, 22, 24, 25, 27, 30, 47, 50, 52, 56)
- LEFT JOIN — used specifically to preserve 'no match' rows that carry operational meaning: reserves with no animals (Case 3), animals with no GPS logs (Case 36), orphaned GPS logs (Case 48), reserves with no recent activity (Case 65), and animals with no latest reading (Case 70)
- RIGHT JOIN — used once, combined with UNION and a LEFT JOIN, to emulate a full outer join for the audit list in Case 46
- Self joins — not present in the source SQL

F. Advanced SQL

- CTEs (WITH ...) — Cases 8, 10, 32, 49, 52
- Subqueries — Cases 8, 19, 20 (in derived logic), 35, 55, 60, 72
- Correlated subqueries — Cases 55, 60, 70 (finding each animal's own latest log)
- CASE expressions — Cases 2, 6, 21, 23, 26, 40, 44, 52, 59, 74
- Window functions — AVG() OVER (Case 7, 26), COUNT() OVER (Case 16), MAX() OVER (Case 40), DENSE_RANK() OVER (Cases 49, 66), LEAD() OVER (Case 62), LAG() OVER (Case 68)
- UNION — Case 46

G. Date, String, and Mathematical Functions

Function	Used in
DATE_SUB / DATEDIFF	Cases 8, 11, 20, 24
TIMESTAMPDIFF	Case 19
CURDATE() / NOW()	Cases 8, 20, 24, 28, 29, 32, 64, 65
DAYNAME()	Case 13
MONTH() / YEAR() / QUARTER()	Cases 15, 32, 67
HOUR()	Case 75
UPPER()	Case 9

Function	Used in
TRIM()	Case 54
CONCAT()	Case 30
LENGTH()	Case 5
ROUND()	Cases 24, 33, 43 (partial), 49, 52

H. Transactions

Data/Source Consistency Note

No COMMIT, ROLLBACK, or explicit START TRANSACTION statement appears anywhere in the uploaded SQL file. Per the reporting instructions, this is stated plainly as Not provided in the source data rather than invented. All UPDATE/DELETE/INSERT statements in the source appear to run under default auto-commit behaviour.

I. Indexing

- `Idx_gpslogs_animal_id` — single-column index on `GPSLogs(animal_id)`, created to speed up the very common lookup of a specific animal's full movement history (Case 28).
- `Idx_Animals_reserve_animal` — composite index on `Animals(reserve_id, animal_id)`, created to speed up reserve-scoped animal lookups (Case 53).
- `Idx_gpslogs_animal_date` — composite index on `GPSLogs(animal_id, log_datetime)`, created specifically because the reporting dashboard filters by animal/reserve and date range together (Case 53); a composite index matching the real combined filter is more effective than two separate single-column indexes.
- `SHOW INDEX FROM gpslogs` — a diagnostic query used to inspect whatever indexing is currently in place (Case 73).

J. Security / Access Control

Element	Detail from source SQL
Users created	<code>researcher_username</code> (Case 1, later dropped), <code>Field_researcher</code> (Case 69), <code>lead_veterinarian</code> (Case 71)
GRANT	<code>SELECT + UPDATE</code> and <code>ALL PRIVILEGES</code> granted to <code>researcher_username</code> (Case 1); <code>SELECT</code> granted to <code>Field_researcher</code> on <code>Animals</code> and <code>GPSLogs</code> (Case 69); <code>SELECT + UPDATE</code> granted to <code>lead_veterinarian</code> on <code>Animals</code> (Case 71)
REVOKE	<code>UPDATE</code> privilege revoked from <code>researcher_username</code> (Case 1)
DROP USER	<code>researcher_username</code> removed entirely, fully withdrawing access (Case 1)
SHOW GRANTS	Used to verify the exact privileges assigned to <code>Field_researcher</code> and <code>lead_veterinarian</code> (Cases 69, 71)
Data/Source Consistency Note	
The GRANT statements in Case 1 reference the	

Element	Detail from source SQL
object project_2.Wildtrack — but no table named Wildtrack exists anywhere in the schema (the actual tables are Reserves, Species, Animals, and GPSLogs, inside the Project_2 database). This appears to be a naming slip in the source SQL rather than a real object. It is reported here exactly as written rather than silently corrected.	

9. Analysis of All 75 Business Cases

Every one of the 75 real-world situations described in the source PDF is analyzed below, grouped into the logical categories specified for this report. For each case: the business scenario is restated in plain terms, the SQL concept required is identified, and the real-world conservation meaning is explained — not simply a repeat of the question, but why the situation and its resolution matter operationally to WildTrack. All SQL approaches described are drawn directly from the uploaded SQL file; no query or result has been invented.

A. Data Retrieval & Aggregation (5 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
4	Count distinct species currently monitored, alliance-wide. — Gives leadership one headline number for board-level reporting on programme breadth.	COUNT(DISTINCT ...)
17	Produce a one-line-per-animal field report with species and reserve. — A simple denormalized report field staff can read without needing to understand the underlying schema.	Multiple LEFT JOIN
30	Combine tag code and species name into a single readable label. — Produces a print-ready label for laminated field handouts used by rangers in the field.	CONCAT(), INNER JOIN
37	Count total animals currently tracked, alliance-wide. — The headline number for the programme director's board meeting.	COUNT(*)
57	Count total GPS pings logged since the programme began. — A data-volume metric for understanding overall database and operational scale.	COUNT()

B. Data Quality & Validation (3 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
5	Flag reserve names longer than 25 characters. — A data-quality/UX check ensuring long names don't break the mobile app's limited display space.	LENGTH(), WHERE
9	Standardize inconsistent tag-code capitalization. — Produces a clean, dashboard-ready tag list despite years of inconsistent manual data entry by different rangers.	UPPER()
54	Clean up reserve names with accidental leading/trailing spaces. — Prevents whitespace artifacts from a bulk import corrupting the printed annual report.	TRIM(), UPDATE

C. Animal & Species Analysis (9 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
6	Bucket animals into Juvenile / Adult / Senior age bands. — Lets the biology team study the age structure of the tracked population at a glance.	CASE expression
10	Tally species population, then join reserve detail for Vulnerable species concentration. — Shows donors and planners exactly where the most at-risk populations are concentrated, guiding resource allocation.	Two CTEs + multiple INNER JOINS
22	Compute average age of all Critically Endangered animals. — Supports the funding proposal narrative around the most at-risk conservation-status tier.	AVG, INNER JOIN, WHERE

#	What the Case Is Asking / Business Meaning	SQL Concept
25	Count tracked individuals per species; highlight species below 10. — A direct conservation-priority signal — species with very small monitored populations may need urgent intervention.	INNER JOIN, GROUP BY, HAVING
34	List every reserve that currently hosts at least one collared individual of each species. — Maps species presence across the reserve network for cross-reserve conservation planning.	WHERE, ORDER BY
47	Find species with at least one individual in a reserve established before 2000. — Connects species presence to reserve history/maturity, useful for understanding long-established conservation strongholds.	INNER JOIN, WHERE
59	Split all animals into three roughly equal age tiers. — Supports three distinct age-specific conservation strategies designed by the biology team.	CASE expression
63	Within each reserve, order animals oldest-first by estimated age. — Helps identify older individuals that may need closer health monitoring.	ORDER BY
72	Identify every animal belonging to the single most-tracked species. — Highlights which species dominates current tracking coverage, useful context for evaluating monitoring balance.	Subquery with GROUP BY, ORDER BY, LIMIT

D. GPS & Movement Analysis (16 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
2	Group GPS pings per animal and flag animals with fewer than 5 total pings. — Surfaces collars that may be malfunctioning or animals that were never properly tracked, so field teams can inspect hardware early.	INNER JOIN, GROUP BY, COUNT, CASE
8	Find each animal's most recent log, then list animals silent for 10+ days. — A two-stage funnel that turns raw pings into an actionable 'gone quiet' watchlist for rangers.	GROUP BY + MAX(), CTE, DATE_SUB
19	For one animal, compute hours between its first and most recent log. — A quick health check on whether a specific collar is still reporting on a normal cadence.	MIN, MAX, TIMESTAMPDIF
24	Find reserves with average animal speed above 15 km/h over the last month. — Elevated average speed across a reserve's animals can indicate fleeing behaviour, stress, or a shared disturbance (e.g. poaching activity).	INNER JOIN, AVG, HAVING, DATE_SUB
26	Flag GPS logs where an animal's speed is notably above its own average. — Personalizes the 'unusual activity' threshold to each animal rather than a fixed programme-wide number, catching bursts specific to that individual's normal behaviour.	Window function — AVG() OVER (PARTITION BY animal_id), CASE
27	Find the reserve of the animal with the single highest recorded speed. — Localizes the most extreme movement event in the dataset to a specific reserve for follow-up investigation.	INNER JOIN, ORDER BY, LIMIT
32	Tally each animal's logs this month, then shortlist those under 3 logs. — A short-term 'gone quiet' early-warning list, complementary to the 10-day check in Case 8.	CTE, COUNT, MONTH/YEAR filter
36	Identify animals with zero GPS pings ever recorded. — A critical operational failure — a collared animal generating no data at all needs immediate hardware or connectivity investigation.	LEFT JOIN, IS NULL

#	What the Case Is Asking / Business Meaning	SQL Concept
41	Find the single highest speed ever recorded by any collar. — The most extreme movement event in the whole dataset — potentially a chase, escape, or predator encounter worth investigating.	MAX()
48	Find GPS logs pointing to an animal ID that no longer exists. — Surfaces orphaned records left behind by a data cleanup — a referential-integrity check on historical data.	LEFT JOIN, IS NULL
49	Score each animal's activity from log count and average speed, then rank reserves by total activity. — Converts raw movement data into a composite, comparable activity ranking across reserves for operational prioritization.	Two CTEs, DENSE_RANK() OVER (ORDER BY ...)
64	Explain what happens when a GPS ping arrives for an unregistered animal ID. — Because GPSLogs.animal_id is a foreign key referencing Animals, the database itself rejects the orphaned insert — this is a conceptual/'explain why' case rather than a corrective query.	Foreign key constraint (referential integrity), demonstration INSERT
65	Find reserves with zero GPS pings in the past 7 days. — Flags a possible communication blackout or a bigger on-the-ground problem at a specific reserve.	LEFT JOIN with date filter in ON clause, IS NULL
70	List every animal with its most recent coordinates, keeping animals with no reports at all. — Ensures animals that have gone completely silent still appear on the list (rather than disappearing), which is itself an important signal.	INNER JOIN x2, LEFT JOIN with correlated subquery
74	Tag each GPS log as Resting / Moving / Fast Movement by speed. — Turns raw speed values into an interpretable activity label, helping spot chase or escape events hidden in raw data.	CASE expression
75	Pull all GPS logs recorded between 6 PM and 6 AM. — Supports a dedicated nocturnal-behaviour study for species that are more active at night.	HOUR()

E. Battery & Collar Monitoring (9 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
7	Smooth battery readings using a 3-reading moving average per reserve. — Filters out momentary sensor noise so battery trends (not single glitchy readings) drive maintenance decisions.	Window function — AVG() OVER (... ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
11	Identify collars fitted more than 2 years ago. — Flags ageing hardware approaching end-of-life so replacement can be scheduled proactively rather than after failure.	DATE_SUB, WHERE
12	Find the single lowest battery level ever recorded. — Points directly at the device most urgently needing a field swap.	MIN()
23	Label each battery reading Critical / Low / Healthy. — Converts a raw percentage into an instantly actionable status field for maintenance teams.	CASE expression
33	Compute the current average battery level across all collars. — A single planning number used to estimate how many battery-replacement field trips are needed this month.	AVG, ROUND
43	Identify reserves with average collar battery below 20%. — Triggers scheduling of an urgent battery-replacement drive at the reserve level rather than one collar at a time.	INNER JOIN, GROUP BY, HAVING

#	What the Case Is Asking / Business Meaning	SQL Concept
45	Correct stored battery readings for one reserve after a firmware recalibration. — The scenario describes a data-correction need, but no corrective SQL statement is present in the file — see Data/Source Consistency Note.	Not provided in the source data
52	Compute reserve-level average battery, then flag reserves below the alliance-wide average. — Benchmarks each reserve against the programme-wide baseline rather than an arbitrary fixed threshold.	Two CTEs, CROSS JOIN, CASE
60	Pull every GPS log belonging to the animal with the lowest-ever battery reading. — Gives the field team the animal's full movement history so they can plan an efficient collar-swap visit.	Correlated subquery

F. Reserve Analysis (8 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
3	Identify reserves with zero collared animals. — Flags a coverage gap — a reserve WildTrack is nominally protecting but not actually monitoring, which matters for donor reporting and patrol planning.	LEFT JOIN, IS NULL
21	Classify reserves as Overcrowded / Balanced / Under-monitored by animals-per-area. — A density metric that helps balance ranger deployment and tracking effort relative to each reserve's physical size.	COUNT, division, CASE
31	Count collared animals per reserve. — Shows where tracking effort is currently concentrated across the reserve network.	GROUP BY, COUNT
35	List animals in the three largest reserves by area. — Concentrates review effort on the reserves with the most physical space to patrol and monitor.	Subquery + ORDER BY + LIMIT (see Data/Source Consistency Note)
40	Surface each reserve's fastest animal without dropping the reserve's other animal rows. — Highlights the standout animal per reserve while preserving the full roster for context, unlike a simple GROUP BY which would collapse the other rows.	Window function — MAX() OVER (PARTITION BY reserve_id), LEFT JOIN, CASE
42	Shortlist reserves currently tracking more than 15 animals. — Identifies reserves that may need additional ranger support given their higher animal load.	GROUP BY, HAVING
44	Build one consolidated per-reserve report: animal count, average age, average battery, and a battery-health flag. — The single query behind the annual conservation report — combining population, demographic, and equipment-health signals per reserve.	LEFT JOIN x2, COUNT(DISTINCT), AVG, CASE
46	Produce a full audit list of every reserve and every animal, including unmatched rows on both sides. — Guarantees nothing is silently dropped from the audit — reserves with zero animals and any orphaned animal record both remain visible.	LEFT JOIN UNION RIGHT JOIN (FULL OUTER JOIN emulation)

G. Date & Time Analysis (4 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
13	Find which day of the week sees the most recorded movement. — Informs ranger patrol scheduling around when animals are naturally most active.	DAYNAME(), GROUP BY, ORDER BY
15	Pull all GPS logs from June of the current year. — Supports a seasonal study of animal behaviour at the onset of monsoon.	MONTH()

#	What the Case Is Asking / Business Meaning	SQL Concept
20	Compute days since each collar was fitted; flag those due for the 180-day check. — Automates a recurring maintenance obligation instead of relying on manual tracking of fitting dates.	DATEDIFF, WHERE
67	Group collaring activity by the exact year and quarter each animal was fitted. — Shows how collaring effort has changed across different phases of the programme's history.	YEAR(), QUARTER(), GROUP BY

H. Advanced SQL / Window Functions (4 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
16	Build a running total of animals collared per species over time. — Visualizes programme growth per species as a cumulative curve rather than a single end total.	Window function — COUNT() OVER (PARTITION BY ... ORDER BY ... ROWS UNBOUNDED PRECEDING)
62	Show each animal's next recorded speed alongside its current one. — Lets analysts see consecutive readings side by side to spot a building pattern (e.g. accelerating movement).	Window function — LEAD() OVER (PARTITION BY animal_id ORDER BY log_datetime)
66	Rank every GPS log by speed, tying equal top speeds fairly. — Surfaces the season's most extreme movement events while ensuring tied speeds share the same rank rather than one being arbitrarily skipped.	Window function — DENSE_RANK() OVER (ORDER BY speed_kmph DESC)
68	Show each animal's previous battery reading alongside the current one. — Catches sudden, unexplained battery drops between consecutive readings — an early warning of hardware failure.	Window function — LAG() OVER (PARTITION BY animal_id ORDER BY log_datetime)

I. Views (3 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
14	Provide a live, reusable reserve-level animal count + average battery health feed. — Gives the dashboard team a single reusable object instead of re-deriving the same joins/aggregates every time.	CREATE VIEW, LEFT JOIN, COUNT(DISTINCT), AVG
55	Provide an always-ready snapshot of each animal's tag, species, reserve, and latest battery reading. — Saves the mobile app team from reconstructing the 'latest reading per animal' logic every time they need it.	CREATE VIEW, correlated subquery for latest log per animal
56	Provide a reusable list of every Critically Endangered animal with reserve/species detail. — Lets donor relations self-serve the same recurring list without re-explaining the schema each time.	CREATE VIEW, INNER JOIN, WHERE

J. Indexing & Performance (3 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
28	Speed up repeated animal-ID lookups on a growing GPSLogs table. — As GPSLogs scales into the thousands (and beyond), an index turns a full table scan into a fast lookup — critical for rangers who query by animal ID constantly.	CREATE INDEX on animal_id

#	What the Case Is Asking / Business Meaning	SQL Concept
53	Speed up dashboard queries that filter by reserve and date range together. — A composite index matching the actual combined filter pattern used by the dashboard is far more effective than two single-column indexes.	Composite CREATE INDEX
73	Check what indexing is currently in place on GPSTags. — A diagnostic step for a new team member/DBA inheriting the system without documentation.	SHOW INDEX

K. Constraints (4 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
18	Prevent battery levels above 100% from ever being saved again. — Turns a manual/UI-level assumption into a database-enforced guarantee, permanently closing that error path.	ALTER TABLE ... ADD CONSTRAINT ... CHECK
39	Prevent two rangers from registering the same tag code for two animals on the same day. — Protects data integrity against a specific real-world race condition — simultaneous entry by rangers in different reserves.	ALTER TABLE ... ADD CONSTRAINT ... UNIQUE (composite)
51	Prevent a new animal record from being saved without a species. — Stops incomplete records at the database level, protecting downstream species-level analysis from silent gaps.	ALTER TABLE ... MODIFY ... NOT NULL
58	Reject a negative estimated age the moment it's submitted. — Enforces a basic biological sanity rule (age cannot be negative) directly in the database.	ALTER TABLE ... ADD CONSTRAINT ... CHECK

L. Data Modification (4 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
29	Record a new collar-fitting date for a re-collared animal. — Keeps operational records accurate when an animal thought dead is confirmed alive and re-equipped.	UPDATE ... SET ... WHERE
38	Capture which manufacturer supplied each animal's collar going forward. — Extends the schema to support future manufacturer-level reliability analysis (e.g. which supplier's collars fail most).	ALTER TABLE ... ADD COLUMN
50	Remove GPS logs tied specifically to decommissioned first-generation collars. — Cleans clutter from a legacy hardware generation without touching any other animal's history.	DELETE ... INNER JOIN ... WHERE
61	Correct an entire reserve's animals that were logged under the wrong reserve. — Fixes a systemic data-entry mistake in one sweep instead of editing every animal individually.	UPDATE ... SET ... WHERE (bulk correction)

N. Security & User Permissions (3 cases)

#	What the Case Is Asking / Business Meaning	SQL Concept
1	Withdraw a departed researcher's database access immediately. — Ends unauthorized access risk the moment fieldwork ends, protecting sensitive location data on endangered animals from a former team member.	CREATE USER, GRANT, REVOKE, DROP USER
69	Give a new field researcher read-only access to animal and GPS data. — Enforces least-privilege access — researchers can view but never alter operational records.	CREATE USER, GRANT SELECT

#	What the Case Is Asking / Business Meaning	SQL Concept
71	Give the lead veterinarian the ability to update animal health details. — Differentiates the vet's access level from ordinary field staff, matching real operational responsibility.	CREATE USER, GRANT SELECT, UPDATE

10. Detailed SQL Technique Explanation

The most important SQL techniques demonstrated in the project are explained below with the exact query from the uploaded SQL file, a line-by-line explanation, and the business purpose it serves.

GROUP BY + HAVING

Case 25 — species with fewer than 10 tracked individuals

```
select s.species_name,
count(a.animal_id) as Total_animal
from species s
inner join animals a
on s.species_id = a.species_id
group by s.species_name
having count(a.animal_id) <10;
```

Line-by-line explanation

- INNER JOIN connects each species to its collared animals.
- GROUP BY s.species_name collapses the result to one row per species.
- COUNT(a.animal_id) counts how many collared animals fall into each species group.
- HAVING filters on the aggregated count itself — something WHERE cannot do, because WHERE runs before grouping.

Business purpose

This directly implements the conservation-priority signal requested in Case 25: species whose tracked population has fallen below a meaningful threshold (10 individuals) are flagged for attention.

INNER JOIN

Case 30 — combining tag code and species name into a field label

```
select
concat (tag_code,'-', species_name) as Animal_lable
from Animals a
inner join Species s
on a.species_id = s.species_id;
```

Line-by-line explanation

- INNER JOIN returns only animal rows that have a matching species record — appropriate here since every animal must belong to a valid species.
- CONCAT() combines two columns from the joined result into a single readable string.

Business purpose

Produces a print-ready label ('TAGCODE-SpeciesName') for laminated field handouts rangers carry in the field.

LEFT JOIN (preserving unmatched rows)

Case 3 — reserves with zero collared animals

```
select r.reserve_id,r.reserve_name
from reserves r
left join animals a
on r.reserve_id = a.reserve_id
where a.animal_id is null;
```

Line-by-line explanation

- LEFT JOIN keeps every reserve row even when there is no matching animal.
- For reserves with no animals, all Animals columns come back NULL.
- WHERE a.animal_id IS NULL isolates exactly those unmatched reserves.

Business purpose

This is the standard SQL pattern for finding 'the parent with no children' — here it flags a real coverage gap the programme director needs to know about.

CTE (WITH clause)

Case 8 — animals silent for more than 10 days

```
with RecentLogs as (
select a.animal_id,
max(g.log_datetime) as Recent_log_date
from animals a
inner join gpslogs g
on a.animal_id = g.animal_id
group by a.animal_id)
select animal_id,Recent_log_date
from RecentLogs
where Recent_log_date < date_sub(curdate(),interval 10 day);
```

Line-by-line explanation

- The CTE RecentLogs first computes each animal's single most recent log timestamp.
- The outer query then filters that intermediate result for dates older than 10 days ago.
- Splitting the problem into a named intermediate step makes the two-stage logic in the business question directly readable in the SQL itself.

Business purpose

Produces the 'gone quiet' watchlist rangers use to prioritize which collars need a physical check.

Subquery

Case 60 — GPS history of the animal with the lowest-ever battery reading

```
select *
from gpslogs
where animal_id =
(select animal_id
from gpslogs
order by battery_level_pct asc
limit 1);
```

Line-by-line explanation

- The inner subquery finds the single `animal_id` associated with the lowest `battery_level_pct` in the whole `GPSLogs` table.
- The outer query then pulls every log belonging to that one animal.

Business purpose

Gives the field team the animal's complete movement history so a battery-swap visit can be planned around where the animal actually is.

CASE expression

Case 23 — battery status labelling

```
select animal_id,battery_level_pct,
case
when battery_level_pct < 15 then 'Critical'
when battery_level_pct between 15 and 40 then 'Low'
else 'Healthy'
end as Battery_level_status
from gpslogs;
```

Line-by-line explanation

- CASE evaluates each row's `battery_level_pct` against ordered conditions.
- The first matching WHEN branch determines the label; ELSE catches everything above 40%.

Business purpose

Converts a raw numeric percentage into an instantly actionable status field field teams can scan without doing mental arithmetic.

RANK / DENSE_RANK

Case 66 — ranking GPS logs by speed without unfairly skipping ties

```
select log_id,animal_id,speed_kmph,
dense_rank() over (order by speed_kmph desc) as Speed_rank
from gpslogs
order by speed_rank;
```

Line-by-line explanation

- `DENSE_RANK() OVER (ORDER BY speed_kmph DESC)` assigns rank 1 to the fastest reading, rank 2 to the next distinct speed, and so on.
- Unlike `RANK()`, `DENSE_RANK()` leaves no gaps in the ranking sequence when two rows tie — exactly the behaviour the business case explicitly requires.

Business purpose

Surfaces the season's most extreme movement events for investigation, with tied top speeds treated fairly rather than one being arbitrarily pushed down the list.

LAG

Case 68 — detecting sudden battery drops

```
select animal_id,log_id,log_datetime,battery_level_pct,
lag(battery_level_pct) over
(partition by animal_id order by log_datetime) as Previous_battery_level
from gpslogs
order by animal_id,log_datetime;
```

Line-by-line explanation

- PARTITION BY animal_id restarts the window separately for each animal.
- ORDER BY log_datetime puts each animal's own logs in chronological order.
- LAG() pulls the battery_level_pct value from the immediately preceding row in that ordered partition.

Business purpose

Placing the previous reading next to the current one lets analysts spot a sudden, unexplained battery drop between two consecutive pings — an early hardware-failure signal.

LEAD

Case 62 — previewing the next speed reading

```
select animal_id,log_datetime,speed_kmph,
lead(speed_kmph) over (partition by animal_id order by log_datetime)
as Next_speed
from gpslogs;
```

Line-by-line explanation

- Mirrors the LAG pattern above but looks forward instead of backward.
- For each animal's chronological log sequence, LEAD() returns the speed recorded in the following row.

Business purpose

Lets analysts see the current and next reading side by side to spot a building pattern, such as an animal steadily accelerating across consecutive pings.

Views

Case 14 — a reusable reserve-level dashboard feed

```
create view Reserve_Animal_Battery_level as
select r.reserve_id,r.reserve_name,
count(distinct a.animal_id) as Animal_count,
avg(g.battery_level_pct) as Average_battery_health
from Reserves r
left join Animals a
on r.reserve_id = a.reserve_id
left join gpslogs g
on a.animal_id = g.animal_id
group by r.reserve_id,r.reserve_name;
```

Line-by-line explanation

- CREATE VIEW saves this multi-join, multi-aggregate query under a permanent name.
- LEFT JOINS (rather than INNER JOINS) ensure reserves with zero animals or animals with zero logs still appear, with NULL/zero aggregates rather than disappearing.

- Once created, the dashboard team can simply `SELECT * FROM Reserve_Animal_Battery_level` instead of rebuilding the joins each time.

Business purpose

Gives the reporting tool a single, always-current object reflecting each reserve's animal count and average collar battery health, without re-deriving the logic manually every time.

Constraints (CHECK, UNIQUE)

Cases 18 & 39 — enforcing valid battery readings and unique daily tag registration

```
alter table Gpslogs
add constraint Check_battery_level
check (battery_level_pct between 0 and 100);

alter table Animals
add constraint Unique_tag_per_day
unique (tag_code, collar_fitted_date);
```

Line-by-line explanation

- The first ALTER TABLE adds a CHECK constraint so any future INSERT/UPDATE attempting a battery value outside 0–100 is rejected by the database itself, not just by application code.
- The second ALTER TABLE adds a composite UNIQUE constraint across two columns together, preventing the same tag code from being registered twice on the same date.

Business purpose

Moves data-quality rules out of hopeful application logic and into guaranteed, database-enforced integrity — closing off two specific, real error paths described in the business scenarios.

Indexes

Case 53 — composite indexing for combined dashboard filters

```
create index Idx_Animals_reserve_animal
on Animals (reserve_id, animal_id);

create index idx_gpslogs_animal_date
on gpslogs (animal_id, log_datetime);
```

Line-by-line explanation

- A composite index on (reserve_id, animal_id) speeds up queries that filter or join on reserve first, then animal.
- A composite index on (animal_id, log_datetime) speeds up queries that filter by a specific animal and a date range together — exactly how the dashboard is described as querying.

Business purpose

Because the dashboard filters by reserve/animal and date range together, a composite index matching that combined access pattern is far more effective than two separate single-column indexes.

Transactions

Not present in the source SQL

```
-- No COMMIT / ROLLBACK / START TRANSACTION statements
-- appear anywhere in the uploaded file.
```

Line-by-line explanation

- Not provided in the source data.

Business purpose

Per the reporting instructions, this is stated plainly rather than fabricated. No transactional wrapping is demonstrated in the source SQL.

GRANT / REVOKE

Case 69 & 71 — role-based access for a field researcher and the lead veterinarian

```
create user 'Field_researcher'@'localhost'
identified by 'FieldResercher@123';
grant select on project_2.animals
to 'Field_researcher'@'localhost';
grant select on project_2.gpslogs
to 'Field_researcher'@'localhost';

create user 'lead_veterinarian'@'localhost'
identified by 'leadveterinarian@123';
grant select, update on project_2.animals
to 'lead_veterinarian'@'localhost';
```

Line-by-line explanation

- CREATE USER provisions a new database account for each role.
- GRANT SELECT gives the field researcher read-only visibility into Animals and GPSLogs — they can look, never change.
- GRANT SELECT, UPDATE gives the lead veterinarian the additional ability to update animal records, reflecting their distinct operational responsibility.

Business purpose

Implements least-privilege, role-based access control: two people with different jobs are given exactly the access their job requires, and no more.

11. Wildlife Conservation Insights

The SQL techniques above translate into concrete, real-world conservation insight. Only claims directly supported by the uploaded source material are made below.

Animals with insufficient GPS data

Case 2's query flags any animal with fewer than 5 total logged pings, and Case 32 separately flags animals with fewer than 3 logs in the current month. Together these give WildTrack both a lifetime and a short-term lens on under-reporting animals whose collars may be malfunctioning.

Animals that have stopped reporting

Case 8 identifies animals whose most recent log is more than 10 days old, and Case 36 identifies the more extreme case — animals with zero GPS logs ever recorded, described in the source as 'a critical operational failure that needs immediate attention.'

Low-battery collars

Case 12 finds the single lowest battery reading ever recorded; Case 23 labels every reading Critical/Low/Healthy; Case 43 finds whole reserves averaging below 20% battery; and Case 33 estimates the average battery level across all collars to plan replacement trips.

Reserves without sufficient animal coverage

Case 3 identifies reserves with zero collared animals at all — a direct coverage gap. Case 21 goes further, classifying every reserve as Overcrowded, Balanced, or Under-monitored relative to its physical area.

Species with low monitored populations

Case 25 highlights species with fewer than 10 tracked individuals — described in the source as 'a conservation priority signal' — and Case 10 shows where the most vulnerable species are concentrated by reserve.

Unusual animal speed / activity

Case 26 flags GPS logs where an animal's speed is notably above its own typical average; Case 24 flags whole reserves averaging above 15 km/h over the last month, described in the source as potentially pointing to 'unusual stress or fleeing behaviour'; and Case 41 surfaces the single highest speed ever recorded, potentially 'a chase, an escape, or a predator encounter.'

Missing GPS records

Case 48 identifies orphaned GPS logs pointing to animal IDs that no longer exist, and Case 64 explains why the foreign-key relationship between GPSLogs and Animals should prevent a ping for an unregistered animal from being silently accepted.

Recent movement patterns

Case 65 identifies reserves with zero GPS pings in the past seven days — described in the source as potentially indicating 'a communication blackout or a bigger problem on the ground' — while Case 70 ensures animals with no recent coordinates still appear on the roster rather than silently disappearing.

Aging collars

Case 11 flags every collar fitted more than two years ago as approaching end-of-life, and Case 20 calculates days since fitting to identify animals due for the routine 180-day battery check.

Reserve-level activity

Case 49 computes a combined activity score per animal from log count and average speed, then ranks reserves by total activity using `DENSE_RANK()` — giving operational leadership a comparative view of which reserves are seeing the most movement.

Conservation priorities

Taken together, Cases 10, 22, and 25 point conservation priority toward Critically Endangered and Vulnerable species with small tracked populations, while Cases 3, 21, and 65 point operational priority toward reserves that are under-monitored or have gone quiet.

12. Data Quality & Data Governance

Issue	Handled by	How
Invalid battery percentages (e.g. 150%)	Case 18	ALTER TABLE ... CHECK (battery_level_pct BETWEEN 0 AND 100)
Negative ages	Case 58	ALTER TABLE ... CHECK (estimated_age >= 0)
Duplicate tag codes on the same day	Case 39	ALTER TABLE ... UNIQUE (tag_code, collar_fitted_date)
Missing species on a new animal record	Case 51	ALTER TABLE ... MODIFY species_id INT NOT NULL
Invalid / unregistered animal IDs on a GPS ping	Case 64	Referential integrity via the FOREIGN KEY on GPSLogs.animal_id
Orphan GPS records	Case 48	LEFT JOIN ... WHERE a.animal_id IS NULL to detect logs pointing at animals that no longer exist
Extra spaces in reserve names	Case 54	TRIM(reserve_name), then UPDATE to persist the cleaned value
Incorrect reserve assignments	Case 61	Bulk UPDATE ... SET reserve_id = ... WHERE reserve_id = ...
Inconsistent tag-code capitalization	Case 9	UPPER(tag_code) for a clean, standardized dashboard list
Unusually long reserve names	Case 5	LENGTH(reserve_name) > 25 flag for the mobile app

Referential integrity is enforced structurally through the two foreign keys in the schema (Animals → Species/Reserves, and GPSLogs → Animals), which is precisely what Case 64 relies on to explain why an unregistered animal ID cannot be quietly accepted onto a GPS ping.

13. Database Security

Removing researcher access (Case 1)

A departed researcher's account is created, granted, then had its update privilege revoked, and finally dropped entirely — the source SQL demonstrates the full lifecycle culminating in complete removal, matching the requirement of immediate, full withdrawal of access.

Read-only researcher access (Case 69)

A new field researcher is granted SELECT only on Animals and GPSLogs — they can view operational data but never modify it, matching a role that should observe but not alter records.

Veterinarian permissions (Case 71)

The lead veterinarian is granted SELECT and UPDATE on Animals specifically — reflecting their distinct clinical responsibility to update animal health-related details, a privilege level ordinary field staff do not have.

Role-based access, generally

Across Cases 1, 69, and 71, the project consistently applies the principle of least privilege: each account receives exactly the access its role requires, nothing more. For a wildlife conservation database this matters because the underlying data — precise animal locations — is sensitive; overly broad access increases the risk that location data could be misused (for example, to assist poaching) rather than protecting the animals it describes.

14. Database Performance

Searching GPS logs by animal ID (Case 28)

Rangers repeatedly search GPSLogs by a specific animal's ID to pull up its movement history, and the source SQL responds with `CREATE INDEX Idx_gpslogs_animal_id ON gpslogs (animal_id)` — turning a query that would otherwise scan the entire table into a fast, targeted lookup.

Composite indexing (Case 53)

Because the dashboard filters by reserve/animal and date range together rather than one column at a time, the source SQL creates two composite indexes — `Animals(reserve_id, animal_id)` and `GPSLogs(animal_id, log_datetime)` — matching the real combined query pattern rather than relying on two separate single-column indexes.

Checking existing indexes (Case 73)

`SHOW INDEX FROM gpslogs` is used as a diagnostic step — useful when, as the scenario describes, 'nobody quite remembers what was set up' after team changes.

Why indexes matter as GPSLogs grows

With 9,760 rows already in GPSLogs and the scenario explicitly describing this table as the fastest-growing part of the system, unindexed lookups by `animal_id` or date range would degrade steadily as more collars and more pings are added. The indexes created in Cases 28 and 53 are a direct, proportionate response to that growth pattern.

15. Data Maintenance

- Updating collar information — Case 29 updates collar_fitted_date when an animal previously assumed dead is re-collared.
- Correcting battery readings — Case 45 describes a firmware recalibration scenario requiring a correction to stored battery readings for one reserve; no corrective SQL statement is present in the source file for this case (Not provided in the source data).
- Updating reserve assignments — Case 61 performs a bulk UPDATE correcting an entire reserve's animals that were mistakenly logged under the wrong reserve, in one sweep rather than animal by animal.
- Deleting obsolete GPS logs — Case 50 uses a DELETE with an INNER JOIN to remove logs tied specifically to decommissioned first-generation collars, and only those logs.
- Updating collar fitting dates — Case 20 calculates days since fitting to flag animals due for their routine 180-day battery check, supporting proactive rather than reactive maintenance.

16. Business Impact

Area	How the database supports it	Related cases
Field patrol planning	Identifies which day of the week sees most movement and which reserves show elevated activity, informing when and where to deploy rangers.	13, 24, 49
Equipment maintenance	Flags low-battery and ageing collars before they fail in the field.	11, 12, 20, 23, 33, 43
Wildlife monitoring	Detects animals that have gone silent or never reported at all.	2, 8, 32, 36, 70
Conservation planning	Surfaces species with dangerously small tracked populations and where vulnerable species are concentrated.	10, 22, 25, 47
Data quality	Cleans inconsistent capitalization, whitespace, and enforces validity at the schema level.	5, 9, 18, 39, 51, 54, 58
Operational decision-making	Consolidates reserve-level health into a single report combining population, age, and battery signals.	44, 52
Donor reporting	Provides reusable views for recurring donor requests, e.g. every Critically Endangered animal.	14, 56
Research activities	Gives researchers and veterinarians appropriately scoped, role-based access to operational data.	1, 69, 71

17. Key Findings

Finding	SQL Technique	Business Meaning	Operational Importance
Animals with fewer than 5 total pings may have malfunctioning collars	GROUP BY, COUNT, CASE	Hardware may be failing quietly	High — direct data-loss risk
Some reserves host zero collared animals	LEFT JOIN / IS NULL	A real coverage gap in the protection network	High — programme credibility
Species with fewer than 10 tracked individuals exist	GROUP BY, HAVING	A direct conservation-priority signal	High — species survival risk
Some reserves average below 20% collar battery	GROUP BY, HAVING, AVG	Site-wide equipment failure risk	High — needs urgent battery drive
Orphaned GPS logs point to animal IDs that no longer exist	LEFT JOIN / IS NULL	A data-cleanup or integrity gap	Medium — undermines report accuracy
Composite index needed for combined reserve + date filters	CREATE INDEX (composite)	Dashboard performance depends on matching query patterns	Medium — user experience / scalability
Role-based access separates researchers, vets, and field staff	GRANT / REVOKE	Least-privilege protects sensitive location data	High — security and animal safety

18. Technical Skills Demonstrated

- SQL — DDL, DML, and analytical querying across a realistically sized dataset (10,000 rows)
- Relational Database Management — a normalized four-table schema with enforced primary/foreign keys
- Data Modeling — one-to-many relationships correctly expressed from Reserves/Species down through Animals to GPSLogs
- Data Cleaning — standardizing capitalization, trimming whitespace
- Data Validation — CHECK and UNIQUE constraints enforcing business rules at the schema level
- Aggregation — COUNT, SUM, AVG, MIN, MAX across multiple business contexts
- Joins — INNER, LEFT, RIGHT (via UNION), each chosen deliberately for the row-preservation behaviour the question required
- CTEs — multi-stage 'compute an intermediate result, then filter/rank on it' queries
- Subqueries — including correlated subqueries for per-animal 'latest record' lookups
- Window Functions — AVG, COUNT, MAX, DENSE_RANK, LAG, LEAD applied over partitions and ordered frames
- Views — reusable objects created for dashboard, donor-relations, and mobile-app consumers
- Indexing — single-column and composite indexes matched to actual query patterns
- Database Security — user creation, GRANT/REVOKE, and least-privilege role design
- Business Analysis — translating 75 unlabeled, plain-language operational situations into the correct SQL technique in each case

19. Project Challenges

- Handling large GPS log volumes — GPSLogs holds 9,760 rows, by far the largest table, requiring indexing strategy rather than brute-force scanning (Cases 28, 53, 73).
- Identifying missing data — distinguishing animals that are merely quiet (Case 8, 32) from animals that have never reported at all (Case 36) required two different queries rather than one.
- Maintaining referential integrity — ensuring GPS logs cannot silently attach to non-existent animals (Case 64) and detecting historical orphans left behind after cleanup (Case 48).
- Detecting abnormal animal behaviour — defining 'unusual' relative to each animal's own typical speed (Case 26) rather than one fixed threshold for every species.
- Managing database permissions — designing three distinct access levels (researcher, field researcher, lead veterinarian) that map to three real operational roles (Cases 1, 69, 71).
- Maintaining query performance — recognizing when a single-column index is enough (Case 28) versus when a composite index matching a real combined filter is required (Case 53).

20. Learning Outcomes

- Writing practical, production-style SQL rather than isolated syntax exercises.
- Translating plain-language business problems — deliberately unlabeled in the source PDF — into the correct SQL technique.
- Applying advanced SQL: CTEs, correlated subqueries, and window functions (LAG, LEAD, DENSE_RANK) in realistic operational contexts.
- Working with multi-table relational data spanning geography (reserves), taxonomy (species), individuals (animals), and time-series events (GPS logs).
- Thinking about data quality as an ongoing discipline — capitalization, whitespace, and validity rules enforced at the schema level, not just cleaned up after the fact.
- Thinking about database security as role design, not a single on/off switch — three genuinely different access levels for three genuinely different jobs.
- Thinking about database performance proactively — indexing chosen to match actual query patterns rather than added reactively after a slowdown.
- Converting SQL output into real conservation insight — from a raw battery percentage to a maintenance priority, from a raw speed value to a possible poaching signal.

21. Interview Talking Points

30-second explanation

"I built a relational database and SQL analysis project for a fictional wildlife conservation programme called WildTrack. It's a four-table schema — reserves, species, animals, and GPS logs — holding 10,000 records, and I wrote SQL to solve 75 real operational problems: things like finding animals whose collars have gone silent, flagging low-battery devices, ranking the most extreme movement events, and setting up role-based database access for researchers and veterinarians."

1-minute explanation

"The project simulates the database behind a wildlife conservation programme that tracks endangered species with GPS collars. I designed the schema — Reserves, Species, Animals, and GPSLogs — with proper primary and foreign keys, then solved 75 unlabeled real-world scenarios using SQL: everything from basic aggregation and joins, to CTEs and correlated subqueries, to window functions like LAG, LEAD, and DENSE_RANK for movement and battery-trend analysis. I also built views for recurring dashboard and donor-relations requests, added indexes once I identified the actual query patterns rangers and dashboards use, and implemented role-based security so a field researcher gets read-only access while a lead veterinarian can update animal health data. The point of the project was to practice turning a business question into the right SQL technique, not just writing syntax in isolation."

2-minute detailed explanation

"WildTrack Conservation Alliance is a scenario-based project simulating the database behind a real wildlife conservation programme operating across a network of forest reserves. The schema has four tables: Reserves, Species, Animals, and GPSLogs, connected by foreign keys — every animal belongs to one species and one reserve, and every GPS log belongs to one animal. Altogether it's 10,000 rows, with GPSLogs alone holding nearly 9,800 rows since collars ping every few hours.

The core of the project is 75 real operational situations, deliberately given to me unlabeled, so the first step each time was figuring out what SQL technique the situation actually called for. Some were straightforward — count distinct species, find the lowest battery reading ever recorded. Others needed CTEs to compute an intermediate result before filtering on it, like finding animals logged fewer than three times this month. I used window functions like LAG and LEAD to compare an animal's current and previous or next reading side by side, and DENSE_RANK to rank GPS logs by speed without unfairly skipping tied values.

Beyond querying, I worked on the operational side of the database: adding CHECK and UNIQUE constraints so invalid battery percentages or duplicate tag codes are rejected outright, building indexes matched to how the dashboard actually filters data — by reserve and date range together — and setting up three different access levels reflecting three real jobs: a departing researcher losing all access, a new field researcher with read-only access, and a lead veterinarian with the added ability to update animal health records. The goal throughout was treating this like a live production database supporting real conservation decisions, not an isolated set of SQL exercises."

10 likely SQL interview questions based on this project — with strong answers

1. Why did you use a LEFT JOIN instead of an INNER JOIN to find reserves with no animals?

An INNER JOIN only returns rows where both tables have a match, so a reserve with zero animals would simply disappear from the result. A LEFT JOIN keeps every reserve row regardless of match, filling in NULLs on the Animals side when there's no match — then WHERE a.animal_id IS NULL isolates exactly the reserves with no matching animal at all.

2. What's the difference between WHERE and HAVING, and where did you use HAVING in this project?

WHERE filters individual rows before grouping happens; HAVING filters groups after aggregation. In the species case, I needed to filter on COUNT(animal_id) < 10 — an aggregated value that doesn't exist until after GROUP BY runs — so that had to be HAVING, not WHERE.

3. Explain the difference between RANK, DENSE_RANK, and ROW_NUMBER.

ROW_NUMBER() always assigns a unique, sequential number even to ties. RANK() gives tied rows the same rank but then skips the next rank number(s). DENSE_RANK() also gives tied rows the same rank but does not skip any subsequent rank — which is exactly why I used DENSE_RANK for the top-speed ranking, since the case specifically required tied top speeds to share a rank fairly.

4. When would you use a CTE instead of a subquery?

A CTE names an intermediate result up front with WITH, which made the two-stage logic in cases like 'find each animal's most recent log, then filter for anything older than 10 days' much more readable than nesting the same logic as an inline subquery, especially once I needed to reference that intermediate result more than once.

5. What's a correlated subquery, and where did you need one?

A correlated subquery references a column from the outer query, so it effectively runs once per outer row rather than once overall. I used one to find each animal's own latest GPS log — the inner query has to reference that specific animal's ID from the outer row to compute the right MAX(log_datetime) for that animal alone.

6. Why add a composite index instead of two single-column indexes?

Because the dashboard filters by animal/reserve and date range together, not separately. A composite index on (animal_id, log_datetime) lets the database satisfy both filter conditions from a single index lookup, which is more efficient than combining two separate single-column indexes.

7. How do CHECK constraints differ from application-level validation?

Application-level validation only stops bad data if every piece of code that writes to the table remembers to check it. A CHECK constraint enforces the rule inside the database itself — for example battery_level_pct BETWEEN 0 AND 100 — so it's guaranteed no matter what inserts or updates the data, including future code I haven't written yet.

8. Explain the role of foreign keys in this schema.

Foreign keys enforce referential integrity: Animals.species_id and Animals.reserve_id must reference real rows in Species and Reserves, and GPSLogs.animal_id must reference a real row in Animals. This is exactly why a GPS ping for an unregistered animal ID should be rejected by the database rather than silently accepted.

9. Why use window functions instead of a self-join for LAG/LEAD-style comparisons?

A self-join to compare each row to the 'next' or 'previous' row for the same animal would require matching on some ordering condition, which gets complicated and expensive at scale. Window functions like LAG() and LEAD() do this natively, partitioned by animal and ordered by time, without needing to join the table to itself at all.

10. Why did you design three different database roles instead of one shared account?

Because the three roles have genuinely different responsibilities: a field researcher should only ever read data, a lead veterinarian needs to update animal health records specifically, and a departing researcher needs every privilege removed immediately. Granting the same broad access to everyone would violate least privilege and unnecessarily expose sensitive location data that, if misused, could put endangered animals at risk.

22. Resume Project Description

2-line resume description

Designed and queried a 4-table, 10,000-record relational database simulating a wildlife GPS-tracking programme; solved 75 real-world operational scenarios using joins, CTEs, window functions, views, indexing, and role-based security.

3–4 bullet resume version

- Designed a normalized 4-table SQL schema (Reserves, Species, Animals, GPSLogs) with primary/foreign keys and CHECK/UNIQUE constraints across 10,000 records.
- Translated 75 unlabeled, real-world business scenarios into SQL solutions using joins, CTEs, subqueries, and window functions (LAG, LEAD, DENSE_RANK).
- Built reusable views for recurring dashboard and donor-reporting needs, and added indexes matched to actual dashboard query patterns.
- Implemented role-based database security (GRANT/REVOKE) reflecting three distinct operational roles: researcher, field staff, and veterinarian.

GitHub project description

WildTrack Conservation Alliance — SQL project simulating the operational database behind a multi-country wildlife GPS-tracking programme. Includes full schema DDL, sample data (10,000 rows), and SQL solutions to 75 real-world business scenarios covering data retrieval, data quality, animal/species/reserve analysis, window functions, views, indexing, and database security.

LinkedIn project description

Built a complete SQL project around a wildlife conservation database — 4 connected tables, 10,000 records, and 75 real operational scenarios ranging from flagging animals with malfunctioning GPS collars to setting up role-based access for field researchers and veterinarians. Hands-on practice translating everyday business questions into joins, CTEs, window functions, views, indexing, and security — the kind of SQL that supports real operational decisions, not just isolated exercises.

23. Conclusion

The WildTrack Conservation Alliance project demonstrates the use of SQL as the operational backbone of a real-world system, not as a series of disconnected exercises. Each of the 75 business cases began as a plain-language operational problem — a researcher leaving, a collar going quiet, a reserve possibly under-monitored — and required first understanding what the situation was actually asking before writing a single line of SQL.

Across the project, the work spans the full lifecycle a relational database actually needs in production: schema design with enforced constraints, everyday querying and aggregation, advanced analytical techniques like CTEs and window functions, reusable views for recurring stakeholder requests, targeted indexing based on real query patterns, and role-based security reflecting genuine operational responsibilities. The result is a body of work that speaks to both SQL technical proficiency and the ability to reason about a real business domain — exactly the combination a Data Analyst or Data Scientist role requires.

24. Appendix

A. Table Summary

Table	Rows	Primary Key	Foreign Keys
Reserves	15	reserve_id	—
Species	25	species_id	—
Animals	200	animal_id	species_id → Species; reserve_id → Reserves
GPSLogs	9,760	log_id	animal_id → Animals

B. SQL Concepts Summary

DDL (CREATE/ALTER TABLE, VIEW, INDEX, USER) · DML (INSERT, UPDATE, DELETE, SELECT) · Constraints (PK, FK, NOT NULL, UNIQUE, CHECK) · Aggregation (COUNT, SUM, AVG, MIN, MAX, DISTINCT) · Joins (INNER, LEFT, RIGHT via UNION) · CTEs · Subqueries (including correlated) · CASE expressions · Window functions (AVG/COUNT/MAX OVER, DENSE_RANK, LAG, LEAD) · Date/String/Math functions · Views · Indexing (single-column and composite) · GRANT/REVOKE security. Transactions (COMMIT/ROLLBACK) were not present in the source SQL.

C. All 75 Case Numbers with SQL Concepts

#	SQL Concept	Category
1	CREATE USER, GRANT, REVOKE, DROP USER	Security & User Permissions
2	INNER JOIN, GROUP BY, COUNT, CASE	GPS & Movement Analysis
3	LEFT JOIN, IS NULL	Reserve Analysis
4	COUNT(DISTINCT ...)	Data Retrieval & Aggregation
5	LENGTH(), WHERE	Data Quality & Validation
6	CASE expression	Animal & Species Analysis
7	Window function — AVG() OVER (... ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)	Battery & Collar Monitoring
8	GROUP BY + MAX(), CTE, DATE_SUB	GPS & Movement Analysis
9	UPPER()	Data Quality & Validation
10	Two CTEs + multiple INNER JOINS	Animal & Species Analysis
11	DATE_SUB, WHERE	Battery & Collar Monitoring

#	SQL Concept	Category
12	MIN()	Battery & Collar Monitoring
13	DAYNAME(), GROUP BY, ORDER BY	Date & Time Analysis
14	CREATE VIEW, LEFT JOIN, COUNT(DISTINCT), AVG	Views
15	MONTH()	Date & Time Analysis
16	Window function — COUNT() OVER (PARTITION BY ... ORDER BY ... ROWS UNBOUNDED PRECEDING)	Advanced SQL / Window Functions
17	Multiple LEFT JOIN	Data Retrieval & Aggregation
18	ALTER TABLE ... ADD CONSTRAINT ... CHECK	Constraints
19	MIN, MAX, TIMESTAMPDIF	GPS & Movement Analysis
20	DATEDIFF, WHERE	Date & Time Analysis
21	COUNT, division, CASE	Reserve Analysis
22	AVG, INNER JOIN, WHERE	Animal & Species Analysis
23	CASE expression	Battery & Collar Monitoring
24	INNER JOIN, AVG, HAVING, DATE_SUB	GPS & Movement Analysis
25	INNER JOIN, GROUP BY, HAVING	Animal & Species Analysis
26	Window function — AVG() OVER (PARTITION BY animal_id), CASE	GPS & Movement Analysis
27	INNER JOIN, ORDER BY, LIMIT	GPS & Movement Analysis
28	CREATE INDEX on animal_id	Indexing & Performance
29	UPDATE ... SET ... WHERE	Data Modification
30	CONCAT(), INNER JOIN	Data Retrieval & Aggregation
31	GROUP BY, COUNT	Reserve Analysis
32	CTE, COUNT, MONTH/YEAR filter	GPS & Movement Analysis
33	AVG, ROUND	Battery & Collar Monitoring
34	WHERE, ORDER BY	Animal & Species Analysis
35	Subquery + ORDER BY + LIMIT (see Data/Source Consistency Note)	Reserve Analysis

#	SQL Concept	Category
36	LEFT JOIN, IS NULL	GPS & Movement Analysis
37	COUNT(*)	Data Retrieval & Aggregation
38	ALTER TABLE ... ADD COLUMN	Data Modification
39	ALTER TABLE ... ADD CONSTRAINT ... UNIQUE (composite)	Constraints
40	Window function — MAX() OVER (PARTITION BY reserve_id), LEFT JOIN, CASE	Reserve Analysis
41	MAX()	GPS & Movement Analysis
42	GROUP BY, HAVING	Reserve Analysis
43	INNER JOIN, GROUP BY, HAVING	Battery & Collar Monitoring
44	LEFT JOIN x2, COUNT(DISTINCT), AVG, CASE	Reserve Analysis
45	Not provided in the source data	Battery & Collar Monitoring
46	LEFT JOIN UNION RIGHT JOIN (FULL OUTER JOIN emulation)	Reserve Analysis
47	INNER JOIN, WHERE	Animal & Species Analysis
48	LEFT JOIN, IS NULL	GPS & Movement Analysis
49	Two CTEs, DENSE_RANK() OVER (ORDER BY ...)	GPS & Movement Analysis
50	DELETE ... INNER JOIN ... WHERE	Data Modification
51	ALTER TABLE ... MODIFY ... NOT NULL	Constraints
52	Two CTEs, CROSS JOIN, CASE	Battery & Collar Monitoring
53	Composite CREATE INDEX	Indexing & Performance
54	TRIM(), UPDATE	Data Quality & Validation
55	CREATE VIEW, correlated subquery for latest log per animal	Views
56	CREATE VIEW, INNER JOIN, WHERE	Views
57	COUNT()	Data Retrieval & Aggregation
58	ALTER TABLE ... ADD CONSTRAINT ... CHECK	Constraints
59	CASE expression	Animal & Species Analysis

#	SQL Concept	Category
60	Correlated subquery	Battery & Collar Monitoring
61	UPDATE ... SET ... WHERE (bulk correction)	Data Modification
62	Window function — LEAD() OVER (PARTITION BY animal_id ORDER BY log_datetime)	Advanced SQL / Window Functions
63	ORDER BY	Animal & Species Analysis
64	Foreign key constraint (referential integrity), demonstration INSERT	GPS & Movement Analysis
65	LEFT JOIN with date filter in ON clause, IS NULL	GPS & Movement Analysis
66	Window function — DENSE_RANK() OVER (ORDER BY speed_kmph DESC)	Advanced SQL / Window Functions
67	YEAR(), QUARTER(), GROUP BY	Date & Time Analysis
68	Window function — LAG() OVER (PARTITION BY animal_id ORDER BY log_datetime)	Advanced SQL / Window Functions
69	CREATE USER, GRANT SELECT	Security & User Permissions
70	INNER JOIN x2, LEFT JOIN with correlated subquery	GPS & Movement Analysis
71	CREATE USER, GRANT SELECT, UPDATE	Security & User Permissions
72	Subquery with GROUP BY, ORDER BY, LIMIT	Animal & Species Analysis
73	SHOW INDEX	Indexing & Performance
74	CASE expression	GPS & Movement Analysis
75	HOUR()	GPS & Movement Analysis

D. Important Queries

The most representative queries from the project are reproduced in full in Section 10 (Detailed SQL Technique Explanation): GROUP BY + HAVING (Case 25), INNER JOIN (Case 30), LEFT JOIN (Case 3), CTE (Case 8), Subquery (Case 60), CASE (Case 23), DENSE_RANK (Case 66), LAG (Case 68), LEAD (Case 62), Views (Case 14), Constraints (Cases 18 & 39), Indexes (Case 53), and GRANT/REVOKE (Cases 69 & 71).

E. Limitations Explicitly Visible in the Source

- No COMMIT/ROLLBACK/transaction statements appear anywhere in the SQL file (Section 8, Transactions).

- Case 45 (correcting battery readings after a firmware recalibration) has a business scenario but no accompanying SQL statement in the source file.
- Case 35's query references reserves.area (the actual column is area_sq_km) and joins to a subquery without giving it an explicit alias — reproduced here exactly as written in the source rather than silently corrected.
- Case 1's GRANT statements reference project_2.Wildtrack, an object name that does not correspond to any table defined in the schema (Reserves, Species, Animals, GPSLogs) — reproduced here exactly as written in the source rather than silently corrected.
- Cases 18 and 58 add CHECK constraints (battery_level_pct BETWEEN 0 AND 100; estimated_age >= 0) that were already present at initial table creation — the ALTER TABLE statements in the source are functionally redundant with the original schema, though they are preserved here exactly as written since they demonstrate the ALTER TABLE ... ADD CONSTRAINT syntax explicitly.